



TEMA 068 - Resumen

Entorno de desarrollo PHP

Versión	30.2
Fecha de actualización	11/11/2024



ÍNDICE

1. INTRODUCCIÓN A PHP	3
1.1. DEFINICIÓN	3
2. LENGUAJE PHP	3
2.1. CARACTERÍSTICAS PRINCIPALES	3
3. ARQUITECTURA DE APLICACIONES PHP	5
3.1 PRINCIPALES:	5
4. INTEGRACIÓN CON BASES DE DATOS	6
5. EVOLUCIÓN Y MEJORAS EN PHP 7 Y 8	6
5.1 MEJORAS MÁS RELEVANTES EN LAS VERSIONES 7 Y 8:	6
6. FRAMEWORKS DE PHP	7
6.1 LARAVEL	7
6.2 SYMFONY	8
6.3 CODEIGNITER	8
6.4 ZEND FRAMEWORK/ LAMINAS PROJECT:	8
7. CMS EN PHP	9
7.1 WORDPRESS	9
7.2 JOOMLA	9
7.3 DRUPAL	9
7.4 MAGENTO	9
7.5 PRESTASHOP	9
8. PHP EN APLICACIONES ESCALABLES	10
8.1 PHP EN ARQUITECTURAS DE MICROSERVICIOS	10
8.2 PHP EN APIS	10
9. BUENAS PRÁCTICAS EN PHP	10
9.1 ESTÁNDARES PSR	10
9.2 GESTIÓN DE DEPENDENCIAS	10
9.3 TESTING EN PHP (PHPUNIT)	10



1. Introducción a PHP

1.1. Definición

PHP (Hypertext Preprocessor) es un lenguaje de código abierto, de uso general, **muy** extendido y que se adapta especialmente al desarrollo web.

Fue creado por Rasmus Lerdorf en 1995, actualmente su implementación de referencia es producida por The PHP Group. Su última versión estable es: 8.3.10 (1 de agosto de 2024).

Es un lenguaje interpretado del lado del servidor que puede ser incrustado en HTML (Scripting).

Es de tipo dinámico (no es necesario especificar el tipo de una variable, ya que se determina en tiempo de ejecución).

Su sintaxis deriva de lenguajes como C, Java y Perl.

Es compatible con una amplia variedad de bases de datos (MySQL, PostgreSQL, SQLite...) y multiplataforma. Se integra fácilmente con otros servicios web y tecnologías, como APIs REST, SOAP, y se puede combinar con JavaScript, CSS y HTML.

Dispone de una gran comunidad de desarrolladores y una extensa biblioteca de extensiones y frameworks.

2. Lenguaje PHP

2.1. Características principales

2.1.1 Lenguaje de Scripting del lado del servidor

El código de PHP está enfocado a la programación de scripts que se ejecutan en el servidor (back-end), el cual procesa la petición, genera contenido dinámico y envía la salida al navegador web del cliente en formato HTML. Por tanto, la función principal de PHP es generar dinámicamente contenido HTML, lo que permite crear aplicaciones web interactivas.

El código fuente no es visible para el usuario final, por lo que la lógica de la aplicación queda protegida de ser expuesta.

Además de HTML, también puede crear imágenes, archivos PDF, XML, JSON.

2.1.2 Incrustación de HTML

PHP se puede escribir directamente dentro de código HTML usando las etiquetas de apertura `<?php` y de cierre `?>`. El código PHP dentro de estas etiquetas será interpretado y ejecutado por el servidor antes de que el contenido sea enviado al navegador. Esto permite que el HTML y PHP coexistan en el mismo archivo.

2.1.3 Sintaxis Básica

El código PHP se escribe dentro de un archivo con la extensión `' .php '`.



Etiquetas			
<?php	Apertura	?>	Cierre
Comentarios			
// ó #	Una línea	/* ... */	Varias líneas
Variables			
\$	Declarar variable. Ejemplo: \$nombre = "Pepe";		
Operadores			
Aritméticos	+ suma; - resta; * multiplicación; / división; % módulo; ** potencia;		
Asignación	=	Establece el valor. Ej.: \$a = 3;	
Asignación con suma	+=	Suma un valor al actual. Ej: \$a += 2; // \$a = \$a + 2	
Asignación con resta	-=	Resta un valor al actual. Ej: \$a -= 2; // \$a = \$a - 2	
Otras asignaciones:	*= /= %= .=	Multiplicación, División, Módulo, Concatenación	
Comparación	==, ===, !=, !==, >, <, >=, <=		
Lógicos	&& : AND; : OR; ! : NOT;		
Estructuras de Control			
Condicionales	if, else, elseif		
Bucles	for, while, do...while, foreach		
Funciones	function nombreFuncion(\$parametro) { };		
Funciones Anónimas (Closures)	\$nombreFuncion = function(\$parametro) { };		
Arrays			
Array Indexado	\$nombreArray = ["elemento0", "elemento1", ...];		
Array Asociativo	\$nombreArray = ["elemento0" => valorelemento0, ...];		
Superglobales			
\$_GET	Contiene datos enviados por método GET.		
\$_POST	Contiene datos enviados por método POST.		
\$_SESSION	Contiene datos de sesión		
\$_SERVER	Contiene información del servidor y del entorno de ejecución		
Inclusión archivos			
include 'header.php';	Incluye el archivo header.php		
require 'config.php'	Requiere el archivo config.php		
Manejo de excepciones			
try { // Código que puede lanzar una excepción throw new Exception("Ocurrió un error.");			



```
} catch (Exception $e) {  
    echo "Excepción capturada: " . $e->getMessage();  
}
```

2.1.4 Manejo Automático de Memoria

El manejo automático de memoria en PHP permite que el lenguaje gestione la asignación y liberación de memoria sin intervención directa del desarrollador. PHP asigna memoria automáticamente cuando se crean variables u objetos y utiliza un sistema de conteo de referencias para liberar memoria cuando ya no es necesaria. Además, el garbage collector de PHP maneja ciclos de referencias para evitar fugas de memoria, asegurando un uso eficiente de los recursos durante la ejecución del script.

2.1.5 Casos de Uso de PHP

Algunos ejemplos de casos de uso de PHP son:

- Desarrollo de aplicaciones web dinámicas,
- Gestión de contenidos (CMS),
- Integración con servicios web y APIs,
- Automatización de tareas del servidor

3. Arquitectura de Aplicaciones PHP

3.1 Principales:

3.1.1 Modelo Cliente-Servidor

PHP sigue el modelo cliente-servidor, donde el cliente (navegador web) envía solicitudes al servidor, que procesa estas solicitudes y devuelve una respuesta.

3.1.2 PHP como parte del stack LAMP

El stack LAMP (Linux, Apache, MySQL, PHP) es una arquitectura muy extendida para desarrollar aplicaciones web con PHP.

3.1.3 Patrones de Arquitectura

- MVC (Modelo-Vista-Controlador)
- Arquitectura basada en Componentes o Microservicios

3.1.4 Persistencia de Datos

- Conexión Directa a Bases de Datos
- ORM (Object-Relational Mapping)



3.1.5 Manejo de Sesiones y Estado

Dado que HTTP es un protocolo sin estado, PHP proporciona mecanismos para manejar sesiones y mantener el estado entre solicitudes como: Sesiones en PHP (permite almacenar datos en el servidor) y Cookies (pequeñas piezas de información que se almacenan en el navegador del cliente y se envían al servidor con cada solicitud)

3.1.6 Despliegue y escalabilidad

La arquitectura debe considerar cómo se desplegará la aplicación y cómo escalará para manejar un aumento en la carga de usuarios.

- Despliegue en Servidores Web: Utilizando Apache, Nginx, o servidores en la nube.
- Caché: Uso de sistemas de caché como Memcached o Redis para reducir la carga en la base de datos y acelerar la respuesta.
- Balanceo de Carga: Distribuir las solicitudes entre varios servidores para manejar mejor el tráfico pesado.

4. Integración con Bases de Datos

La integración con bases de datos permite que las aplicaciones gestionen, almacenen y recuperen datos de manera eficiente.

Opciones de Conexión a Bases de Datos en PHP:

- **'mysqli'** (MySQL Improved): extensión que proporciona una interfaz mejorada para interactuar con bases de datos MySQL. Ofrece tanto una interfaz orientada a objetos como una procedimental, y soporta características avanzadas como transacciones y consultas preparadas.
- **PDO** (PHP Data Objects): extensión que proporciona una interfaz de acceso a bases de datos orientadas a objetos y que es compatible con múltiples sistemas de gestión de bases de datos (MySQL, PostgreSQL, SQLite, etc.). PDO soporta consultas preparadas y transacciones, y destaca por su flexibilidad y seguridad.
- Consultas Preparadas: son una característica clave para mejorar la seguridad y el rendimiento de la interacción con bases de datos, previenen la inyección de SQL y contribuyen a la reutilización y eficiencia.
- **'mysqli'** (MySQL Improved): extensión que proporciona una interfaz mejorada para interactuar con bases de datos MySQL. Ofrece tanto una interfaz orientada a objetos como una procedimental, y soporta características avanzadas como transacciones y consultas preparadas.
- **PDO** (PHP Data Objects): extensión que proporciona una interfaz de acceso a bases de datos orientadas a objetos y que es compatible con múltiples sistemas de gestión de bases de datos (MySQL, PostgreSQL, SQLite, etc.). PDO soporta consultas preparadas y transacciones, y destaca por su flexibilidad y seguridad.

5. Evolución y mejoras en PHP 7 y 8

5.1 Mejoras más relevantes en las versiones 7 y 8:

5.1.1 PHP 7



Fue lanzado en diciembre de 2015 y representó una de las actualizaciones más importantes en la historia de PHP:

- **Rendimiento Mejorado (PHPNG - PHP Next Generation):** introdujo un motor de ejecución completamente reescrito, conocido como PHPNG, con mejoras de rendimiento como reducción del consumo de memoria y aumento de la velocidad de ejecución.
- **Operador de Nave Espacial (<=>):** permite realizar comparaciones combinadas en una sola operación.
- **Tipos Escalares y Declaración de Tipo de Retorno:** soporte para la verificación de tipos en las funciones, permitiendo a los desarrolladores especificar el tipo de datos que deben tener los parámetros de entrada y los valores de retorno.
 - Tipos escalares: **int, float, string, y bool.**
 - Strict typing (Tipado estricto): Se puede activar usando `declare(strict_types=1);` al comienzo de un archivo PHP.
- **Manejo de Errores Mejorado** (Excepciones de Errores): Bloques Try- Catch
- **Null Coalesce Operator ('??'):** para el manejo de variables que podrían no estar definidas
- **Mejoras en la Gestión de Memoria y Caching:** eficiencia y rendimiento del OPcache

5.1.2 PHP 8

Lanzado en noviembre de 2020. Mejoras:

- **JIT (Just-In-Time Compilation):** JIT compila partes del código en tiempo de ejecución a código de máquina, lo que puede mejorar aún más el rendimiento en ciertos tipos de aplicaciones, especialmente las que realizan cálculos intensivos (no tanto las de web).
- **Atributos (Attributes):** forma de añadir metadatos a las clases, métodos y funciones de forma similar a las anotaciones en otros lenguajes.
- **Unión de Tipos (Union Types):** permite especificar múltiples tipos de datos para un parámetro o valor de retorno, permitiendo una mayor flexibilidad en las funciones.
- **Operador Null Safe ('?->'):** simplifica el manejo de métodos o propiedades de objetos que podrían ser null.
- **Named Arguments (Argumentos Nombrados):** permite pasar argumentos a las funciones usando el nombre del parámetro, permitiendo mayor claridad y flexibilidad al llamar funciones con muchos parámetros.
- **Expresiones 'match':** que es similar a 'switch', pero más poderosa y menos propensa a errores. 'match' no requiere *break*, admite expresiones de una línea y evalúa de manera estricta (sin conversión de tipos).
- **Mejoras en el Sistema de Tipos**
- **Compatibilidad con Expresiones Regulares Mejorada**

6. Frameworks de PHP

Son herramientas que proporcionan una estructura básica para desarrollar aplicaciones web. Facilitan el desarrollo al ofrecer componentes reutilizables, buenas prácticas, y herramientas que simplifican tareas comunes como la autenticación, manejo de bases de datos, validación de formularios...

6.1 Laravel

Es uno de los más populares y ampliamente utilizados. Su filosofía es desarrollar código PHP de forma elegante y simple. Es de código abierto.



Lanzado en 2011. Última versión estable: 11.18.1 (26 de julio de 2024).

Características Clave:

- Eloquent ORM: Un sistema de mapeo objeto-relacional (ORM) que facilita la interacción con bases de datos mediante objetos PHP.
- Blade: Un potente motor de plantillas que permite construir vistas de manera eficiente.
- Routing sencillo: Maneja rutas de manera simple y flexible.
- Sistemas de autenticación y autorización.
- Queues y Jobs: Manejo de trabajos en segundo plano y colas para mejorar el rendimiento.

Muy útil para: Aplicaciones web complejas, APIs, y proyectos que requieren escalabilidad y flexibilidad.

6.2 Symfony

Potente y flexible, es conocido por su estabilidad y por ser altamente configurable. Diseñado para desarrollar aplicaciones web basado en el patrón Modelo Vista Controlador. Separa la lógica de negocio, la lógica de servidor y la presentación de la aplicación web. Especialmente popular en aplicaciones empresariales y grandes proyectos. Es de código abierto.

Lanzado en 2005. Última versión estable: 6.3.0 (31 de mayo de 2023)

Características clave:

- Componentes reutilizables
- Flexibilidad
- Bundle system: Los "bundles" permiten añadir funcionalidades de manera sencilla y reutilizable.
- Gran comunidad y soporte

6.3 Codeigniter

Ligero y de alto rendimiento que es fácil de aprender y usar. Es conocido por su simplicidad y su enfoque en la velocidad. De código abierto, fue creado por EllisLab y ahora es mantenido por la comunidad de desarrolladores.

Lanzado en 2006. Última versión estable: 4.5.114 (abril de 2024)

Características Clave:

- Ligero: con un núcleo pequeño que no incluye muchas funcionalidades integradas, lo que lo hace rápido y fácil de entender.
- Documentación completa: Una de las mejores documentaciones entre los frameworks PHP.
- Facilidad de uso: Su simplicidad lo hace ideal para principiantes y para proyectos pequeños a medianos.
- No requiere línea de comandos: A diferencia de otros frameworks modernos, muchas configuraciones se pueden realizar directamente en archivos de configuración.

6.4 Zend Framework/ Laminas Project:

6.4.1 PHP 7

El Zend Framework, ahora renombrado como Laminas, es un framework de código abierto orientado a objetos



y altamente modular. Conocido por su enfoque en las mejores prácticas de desarrollo y por ser "enterprise-ready". Zend/Laminas es un framework de uso general con una arquitectura flexible. Lanzado en 2006. Última versión estable: 3.0.0 (28 de junio 2006).

Características Clave:

- Componentes Modulares
- Soporte para MVC y otras arquitecturas
- Alta Configurabilidad
- Documentación Completa
- Seguridad

7. CMS en PHP

Los sistemas de gestión de contenido (CMS) basados en PHP son herramientas que permiten a los usuarios crear, gestionar y modificar contenido en un sitio web sin necesidad de conocimientos avanzados de programación.

7.1 WordPress

El CMS más popular del mundo, originalmente desarrollado como plataforma de blogs. Es capaz de manejar sitios complejos, permite una gran extensibilidad con más de 58.000 plugins disponibles, gran cantidad de temas para personalizar el diseño, facilidad de uso y una comunidad de desarrolladores amplia y activa.

7.2 Joomla

Ampliamente utilizado, conocido por su flexibilidad y capacidad para manejar sitios web de complejidad media a alta. Aunque es un poco más complejo que WordPress, ofrece una gran cantidad de características integradas: soporte multilinguaje, sistema de plantillas, más de 8.000 extensiones, robusto sistema de gestión de usuarios...

7.3 Drupal

CMS de código abierto flexible y capaz de manejar sitios complejos y escalables. Es ideal para desarrolladores que necesitan un alto grado de personalización. Dispone de una gran cantidad de módulos que permiten agregar casi cualquier funcionalidad.

7.4 Magento

Principalmente es una plataforma de comercio electrónico, pero se clasifica también como CMS debido a su robusto sistema de gestión de contenido. Muy adecuado para tiendas online que necesiten una alta personalización y escalabilidad.

7.5 Prestashop

Es una plataforma de comercio electrónico, muy adecuada para pequeñas y medianas empresas. También



cuenta con una amplia variedad de módulos para personalizar y ampliar la tienda.

8. PHP en aplicaciones escalables

8.1 PHP en Arquitecturas de Microservicios

PHP permite crear microservicios independientes que pueden ser desplegados y actualizados sin afectar a otros servicios. Pueden ser escalados horizontalmente. Puede coexistir con otros lenguajes. Es compatible con tecnologías de contenedores como Docker.

8.2 PHP en APIs

Ampliamente utilizado para construir APIs RESTful, para lo que se pueden utilizar frameworks como Laravel, Symfony o Lumen.

PHP soporta múltiples estrategias de autenticación para APIs, como tokens JWT (JSON Web Tokens), OAuth2, y autenticación básica.

También se puede utilizar GraphQL, que es una alternativa a REST que permite a los clientes solicitar sólo los datos que necesitan. PHP soporta GraphQL a través de bibliotecas como webonyx/graphql-php.

9. Buenas prácticas en PHP

9.1 Estándares PSR

PSR (PHP Standards Recommendations) son un conjunto de recomendaciones adoptadas por la comunidad PHP para garantizar la consistencia y calidad del código. Seguir estos estándares ayuda a mantener el código limpio, legible y fácil de mantener, especialmente en equipos de desarrollo.

Los principales son: PSR-1: Reglas Básicas de Codificación, PSR-2: Guía de Estilo de Codificación, PSR-4: Autocarga de Clases.

9.2 Gestión de dependencias

Composer es el gestor de dependencias más popular en PHP. Facilita la instalación, actualización y gestión de bibliotecas y paquetes externos para el proyecto. Permite declarar las bibliotecas que se requieran y automáticamente se encarga de instalar esas dependencias.

9.3 Testing en PHP (PHPUnit)

PHPUnit es el marco de pruebas unitarias más utilizado en PHP. Permite a los desarrolladores escribir pruebas automáticas que aseguren que el código funciona según lo esperado, lo cual es esencial para mantener la calidad y estabilidad del software a medida que se desarrolla.

